

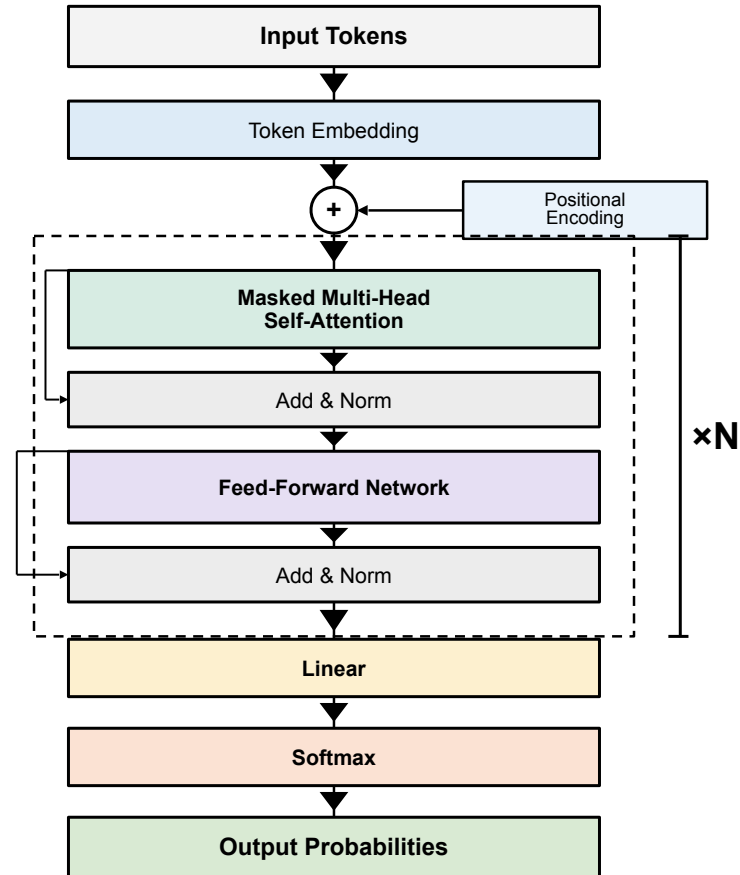
LLMs - Retrieval Augmented Generation

Sriram Ganapathy
Associate Professor, EE, IISc
Co-chief Project Manager, TANUH AI-CoE

LLMs - Recap

Decoder-only transformer architecture - large language models

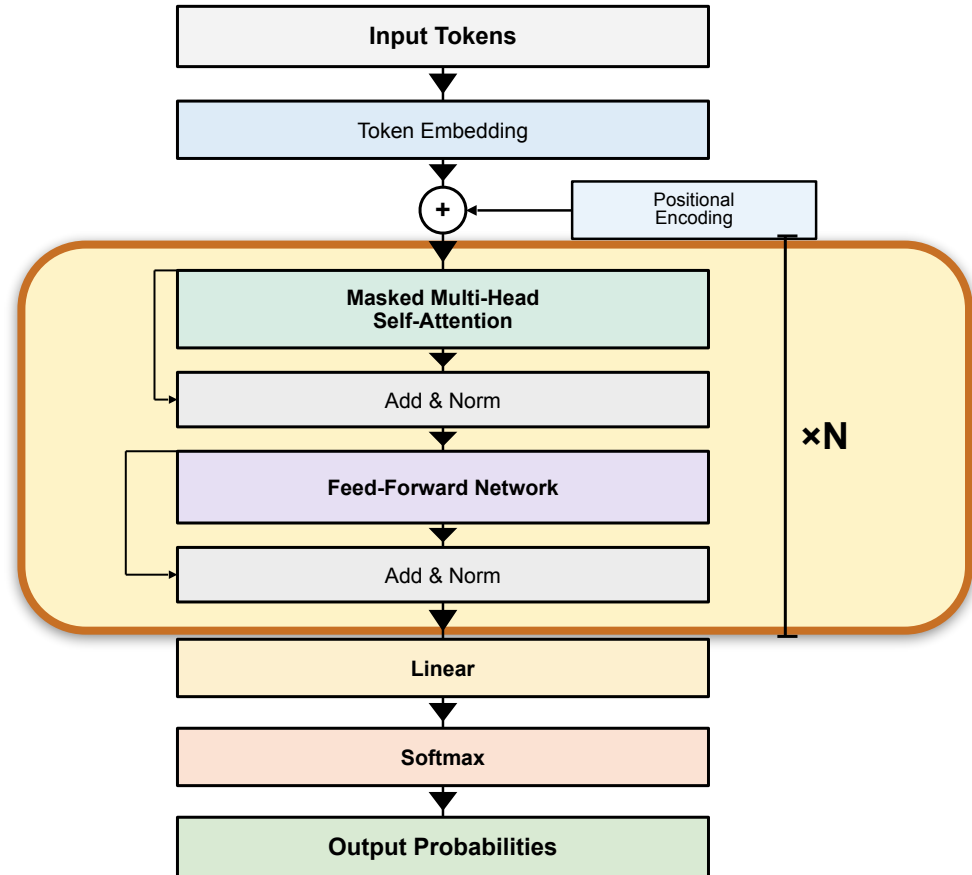
Each of the N decoder blocks applies masked multi-head self-attention and a position-wise feed-forward network,



LLMs - Recap

Parametric memory

- is knowledge that's baked into a model's weights during training
- it's frozen at training time, can't easily be updated without retraining or fine-tuning



LLMs - Recap

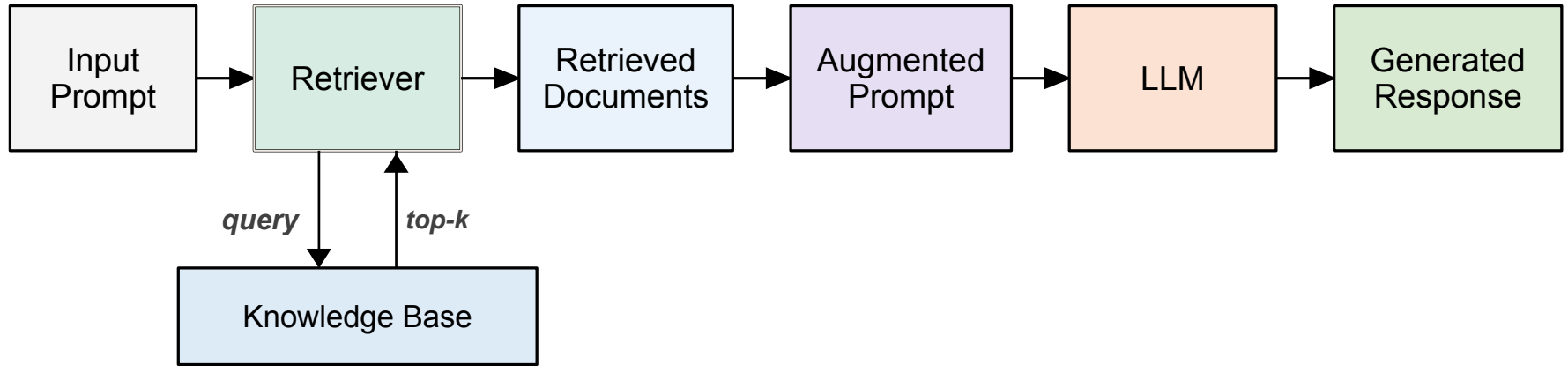
Limitations with parametric memory

- Model is not aware of the constant happenings in the world
- Unable to locate the source of information
- Hallucinations
- Capacity is fixed and finite

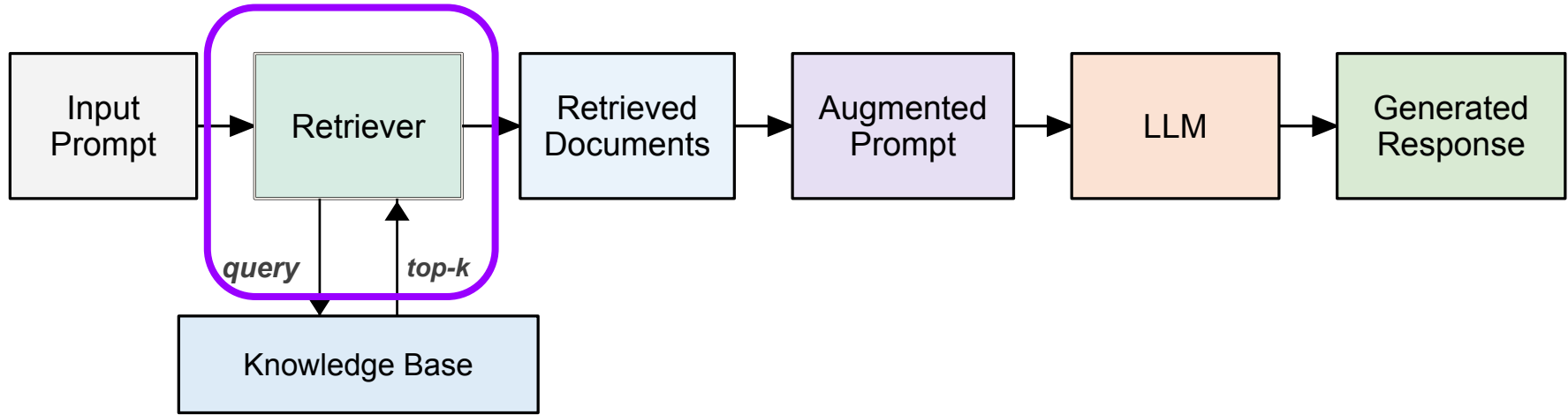
Can we extend to non-parametric memory

- Allowing information update
- Increase capacity
- Reduce hallucinations

What is Retrieval Augmented Generation



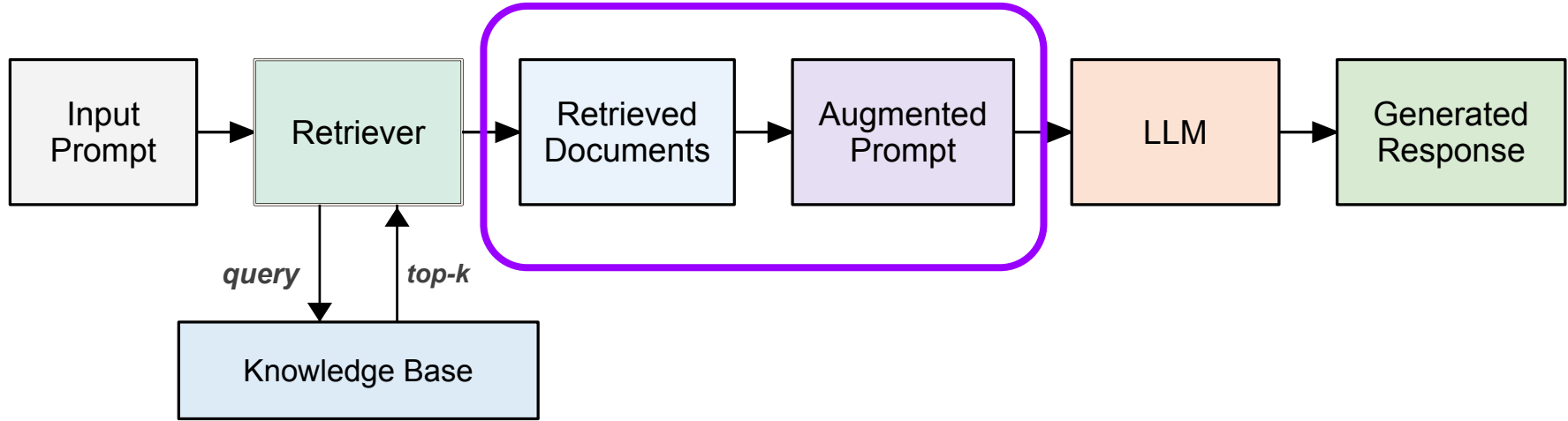
What is Retrieval Augmented Generation



Input - “Generate a poem about Bangalore weather today”

- Retriever is activated (an initial parsing may be present to identify the need of a retriever)

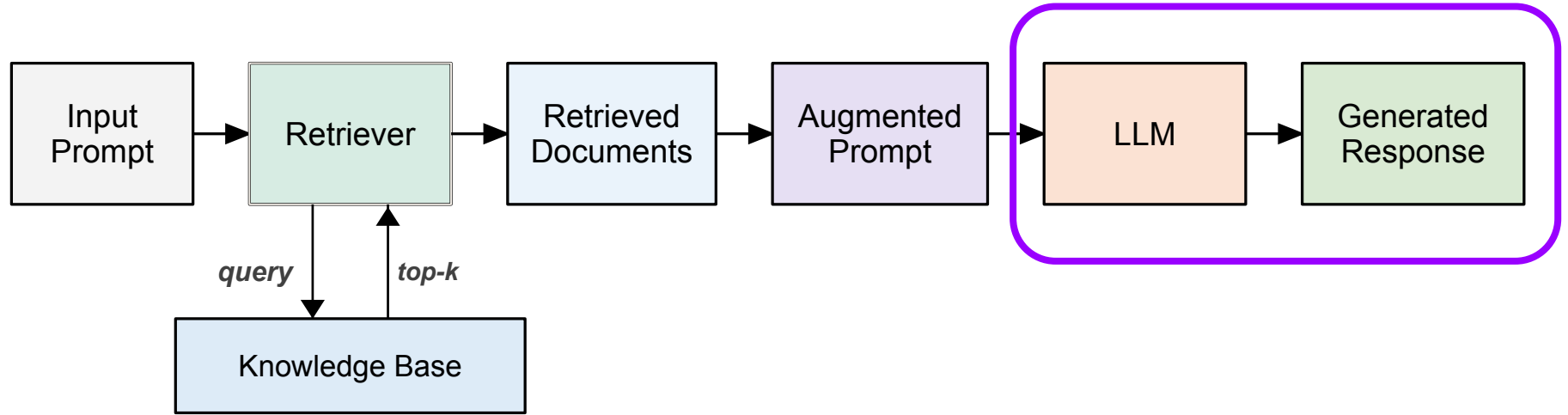
What is Retrieval Augmented Generation



Augmented input

- Retrieved snippets appended to the input query
- Acts the non-parametric memory

What is Retrieval Augmented Generation



Response generation

- Input prompt with context provided by retrieved documents
- “*Bengaluru wears grey silk, the sun a rumor beneath it ...*”

RAG - Fundamentals

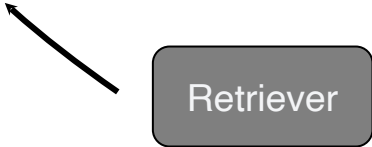
Let **x** denote the input and let **y** denote the output of the model

- Here **d** denotes the document and **C** denotes the corpus.

In practice, the conditioning is approximated with top-**k** documents.

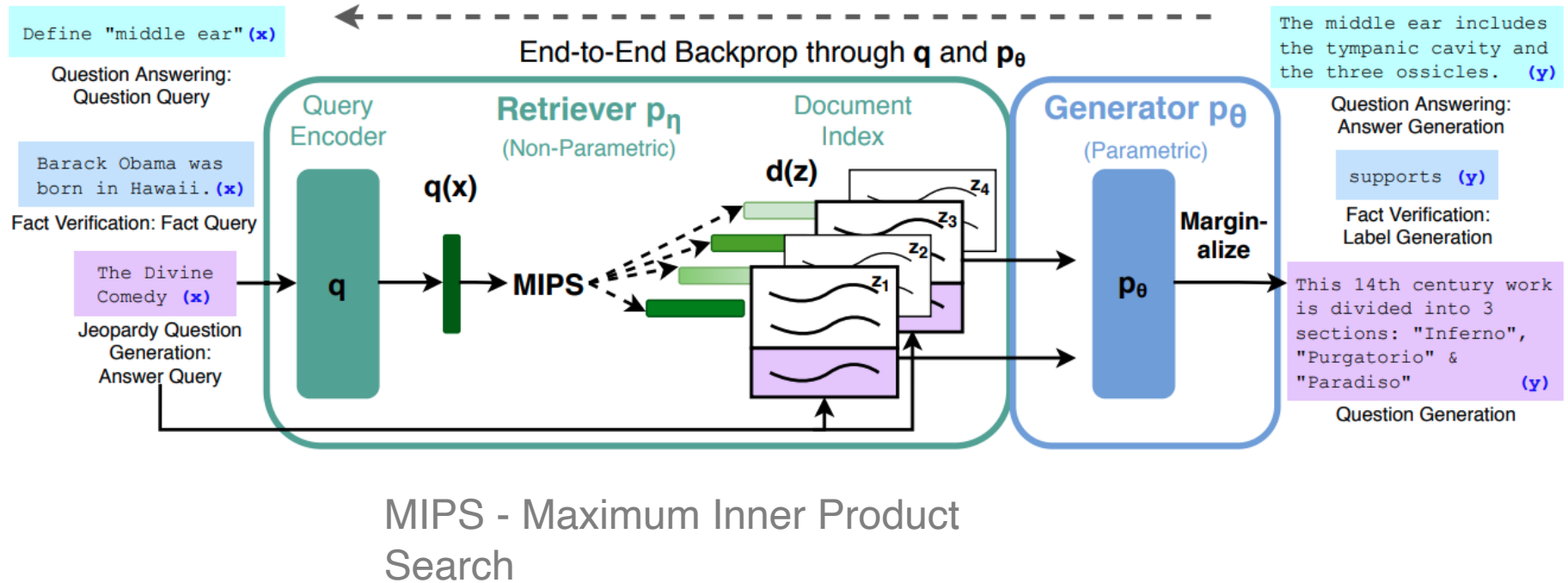
$$P(y|x) = \sum_{d \in \mathcal{C}} P(y|x, d) \cdot P(d|x)$$

$$P(y|x) \approx \sum_{i=1}^k P(y|x, d_i) \cdot P(d_i|x)$$



Retriever

Retriever - In detail

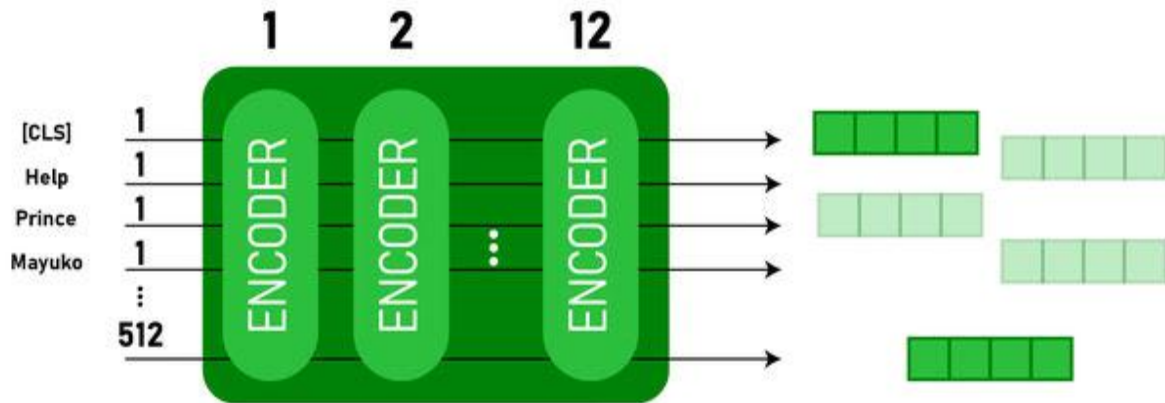


Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474.

Retriever - Query and Document Encoder

Simple BERT Encoder

- A transformer encoder of the tokens in the query
- Entire query is encoded as the average of the token representations
- Passages in the documents can also be encoded using the BERT.



Retriever - MIPS

Inner Product Search

- Both the query **x** and the document **z** are encoded
- A relevance score is the simple inner product between the query and document
- Picking the top-k as the retriever output

$$q(x) = \text{Enc}_q(x) \in \mathbb{R}^h$$

$$d(z) = \text{Enc}_d(z) \in \mathbb{R}^h$$

$$\text{score}(x, z) = q(x)^\top d(z)$$

$$\mathcal{Z}_k(x) = \text{top-}k_{z \in \mathcal{Z}} q(x)^\top d(z)$$

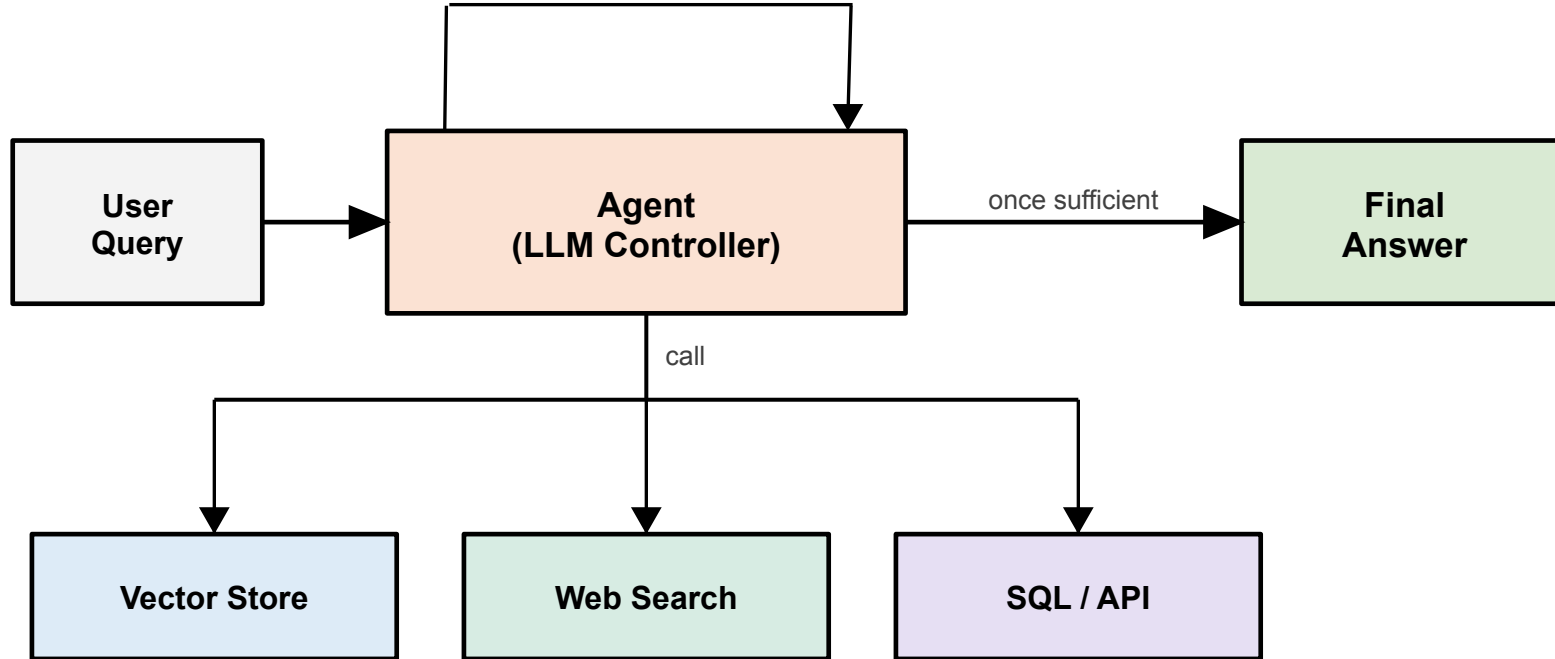
$$p_\eta(z \mid x) = \frac{\exp(q(x)^\top d(z))}{\sum_{z' \in \mathcal{Z}} \exp(q(x)^\top d(z'))}$$

Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474.



What is Agentic AI

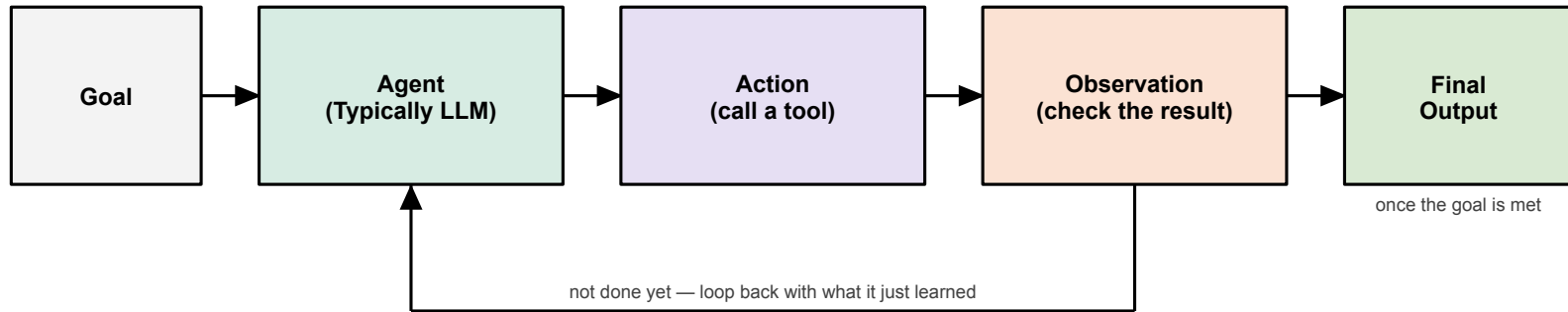
iterate: reformulate query → re-retrieve → re-evaluate



The agent picks whichever tool fits the sub-question, and can call more than one.

What is Agentic AI

An AI that pursues a goal through its own loop of reasoning and action — instead of answering one prompt and stopping.



It chooses its own next step, rather than waiting for the next instruction.

It can act on the world — tools, code, APIs — and react to what comes back.

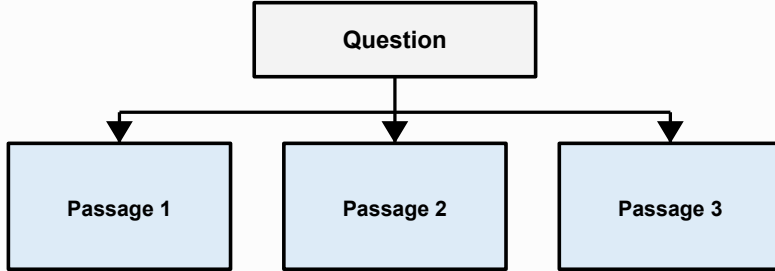
Agentic RAG

Dimension	Standard RAG	Agentic RAG
Retrieval steps	Exactly one pass	One or more passes, as needed
Query handling	Used as-is	Can be reformulated or decomposed
Tool / source choice	Single fixed knowledge base	Agent picks from multiple tools (vector store, web, SQL, APIs)
Stopping condition	Always stops after one retrieval	Agent decides once evidence is sufficient
Latency & cost	Lower — one retrieval + one generation call	Higher — multiple LLM and retrieval calls
Best suited for	Simple, well-scoped factual lookups	Multi-hop, ambiguous, or open-ended questions

Graph RAG

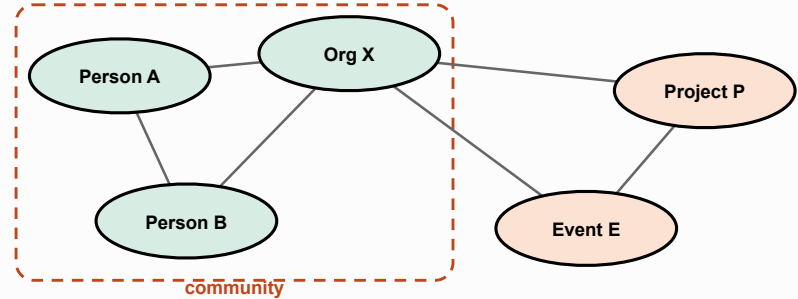
RAG that retrieves from a knowledge graph built out of the corpus — connected entities and summaries of whole clusters of them — instead of a handful of independent text passages.

Standard RAG



The top-k passages most similar to the question. They arrive as a flat list — nothing records how they relate to one another, so questions spanning many documents fall through.

GraphRAG

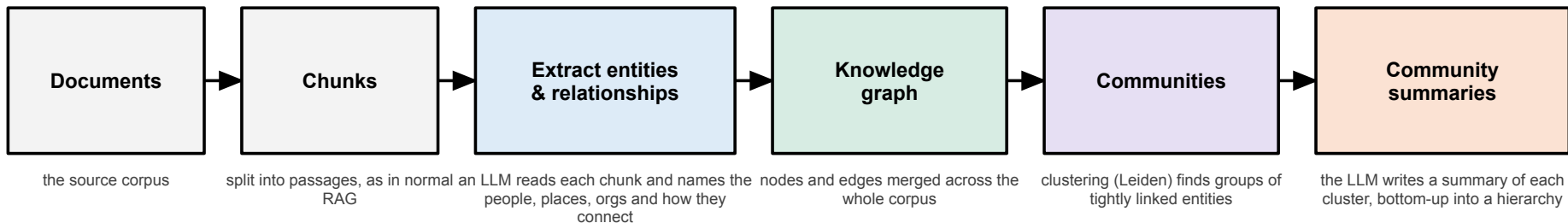


Entities are nodes, the relationships between them are edges, and clusters of the graph get their own LLM-written summary — so the model can retrieve a connected structure, or a summary of the whole corpus.

Graph RAG

A one-time pass in which an LLM reads the corpus and turns it into a graph, which is then clustered and summarized.

Building the graph (indexing)



Indexing is where GraphRAG spends its money: every chunk goes through the LLM once to extract entities, and every community goes through it again to be summarized. Queries afterwards are less expensive — they read the graph, not the corpus.

Agentic RAG

	Standard RAG	GraphRAG
What gets indexed	Text chunks, embedded as vectors	Entities, the relationships between them, and summaries of entity communities
What retrieval returns	The top-k passages most similar to the query	A neighbourhood of the graph (local), or the community summaries (global)
Pros	Pinpoint lookup — “where does it say X?”	Joining facts across documents, and whole-corpus questions — “what are the main themes?”
Cons	Multi-hop questions; anything needing the corpus as a whole	Cost and latency; the graph must be rebuilt as the corpus changes
Index cost	Cheap — embed once	LLM-heavy — every chunk and every community is passed through the model



Sources: Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR. arXiv:2210.03629. | Singh, A. et al. (2025). Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. arXiv:2501.09136.



Thank You

Sriram Ganapathy
sriramg@iisc.ac.in

